Optimizing LoRA Fine-Tuning for Skin Lesion Classification Using Vision Transformers

CS 594: Updated Project Proposal

Gauthami Ghadiyaram Kaushik Sanjay Prabhakar Jinhong Bai

Team repository: `https://github.com/kaushik-3009/Modern-Optimization-Project`

1. Objective and progress. Since the initial proposal, we have downloaded DermaMNIST+, checked split sizes and trainable parameters, and trained a linear probe and rank-4 LoRA on DeiT-Tiny. Both ran for three epochs on CPU with seed 42, batch size 32, AdamW learning rate 5×10^{-4}, unweighted cross-entropy, and decay 0.01 on all trainable parameters. LoRA used $\alpha = 8$ and dropout 0.1. Both checkpoints were selected at epoch 3 by validation loss.

Method	Trainable params.	Val. loss	Test macro-F1	Test bal. acc.
Linear probe	1,351	0.7575	0.2993	0.2716
LoRA ($r = 4$)	38,215	0.4841	0.6212	0.6015

Rank-4 LoRA improved test macro-F1 by 32.19 percentage points over the probe. We will now compare fixed-rank LoRA with group regularization and test subgradient and proximal updates.

2. Data and prediction pipeline. DermaMNIST has 10,015 RGB images in seven lesion classes, derived from HAM10000 [1, 2]. We use the official 224×224 release and 7,007/1,003/2,005 train/validation/test split [3]. Inputs are rescaled to $[0, 1]$ and normalized with channel mean/std 0.5 [4]. We will add training-only flips and rotations [5] and check shapes, ranges, labels, class counts, checksums, duplicates, and lesion overlap. For image x and class $y \in \{0, \dots, 6\}$, pretrained `facebook/deit-tiny-patch16-224` [6] supplies features. We freeze the backbone and train query/value LoRA adapters [7] and a seven-class head; the probe trains only the head. Prediction is $\hat{y} = \arg\max_c p_\Theta(c \mid x)$.

3. Optimization formulation. For each adapted projection ℓ of width d, set

$$W'_\ell = W_{0,\ell} + \frac{\alpha}{r} B_\ell A_\ell, \qquad A_\ell \in \mathbb{R}^{r \times d}, \quad B_\ell \in \mathbb{R}^{d \times r}.$$

With adapters and head $\Theta = \{A_\ell, B_\ell, \theta_h\}$, minimize

$$J(\Theta) = -\frac{1}{N} \sum_{i=1}^{N} w_{y_i} \log p_\Theta(y_i \mid x_i) + \lambda R(\Theta).$$

Here N and n_c are training-set and class counts. We will compare $w_c = 1$ with $w_c = N/(7n_c)$, using training labels only. The rank study will use random A, zero B, $\alpha/r = 1$, and frozen pretrained biases. For $L = 12$, $d = 192$, the trainable count is $4Ldr + 7d + 7$.

4. Group regularization and optimization. We will test whether group regularization can reduce adapter size, building on work on adaptive ranks [8]. Grouping row k of A_ℓ and column k of B_ℓ as $g_{\ell,k}$, we will use $R = \sum_{\ell,k} \|g_{\ell,k}\|_2$ to remove whole components. Subgradient and proximal SGD, without momentum, will start from the same rank-8 weights after two unregularized warmup epochs. Proximal SGD applies group soft-thresholding after the cross-entropy step [9]. An L2 baseline uses $R = \sum_\ell (\|A_\ell\|_F^2 + \|B_\ell\|_F^2)$. These runs use an unpenalized head, zero dropout, and zero optimizer decay; L2 is added to the loss [10].

5. Experimental protocol and evaluation. We will tune the probe separately. At $r = 4$, we will compare AdamW rates $\{3 \times 10^{-4}, 5 \times 10^{-4}\}$ and SGD rates $\{10^{-2}, 10^{-1}\}$ (momentum 0.9), without regularization, then compare ranks $\{2, 4, 8, 16\}$. Class weighting will be tested separately. For group penalties, we will compare both methods at $\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}\}$ with matched rates, weights, and data order. We will compare constant rates with decay as $1/\sqrt{e+1}$ at epoch e after warmup. The search covers 24 configurations, followed by three-seed repeats of selected settings. Training will use effective batch size 32, at most 30 epochs, validation macro-F1 selection, and patience three after five epochs. We will record versions, settings, objectives, component counts, timing, and save/reload checks. We will report test macro-F1, balanced accuracy, and macro one-vs-rest AUROC with mean/std, per-class recall, and confusion matrices. We will compare objective curves, active components, SVD update ranks, and performance at similar parameter counts. After pruning zero components while preserving scaling, we will measure adapter size and inference time, using the same hardware for timing and peak memory.

6. Completion plan.

Week	Activities	Deliverables
1	Finish data checks; implement both update methods and save/reload tests.	Data summary, tested code, run times.
2	Run rank, weighting, penalty, and step-size comparisons.	Loss/sparsity curves, selected settings.
3	Repeat across seeds; evaluate and analyze results.	Models, tables, plots, final report.

References

[1] J. Yang et al. MedMNIST v2. *Scientific Data* 10:41, 2023.

[2] P. Tschandl, C. Rosendahl, and H. Kittler. The HAM10000 dataset. *Scientific Data* 5:180161, 2018.

[3] MedMNIST team. MedMNIST+ release documentation.

[4] Meta AI/Hugging Face. DeiT-Tiny checkpoint and image processor.

[5] R. Szeliski. *Computer Vision: Algorithms and Applications*. 2nd ed., Springer, 2022.

[6] H. Touvron et al. Training data-efficient image transformers & distillation through attention. *Proceedings of ICML*, PMLR 139:10347–10357, 2021.

[7] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

[8] Q. Zhang et al. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. *ICLR*, 2023.

[9] N. Parikh and S. Boyd. Proximal Algorithms. *Foundations and Trends in Optimization* 1(3):123–231, 2014.

[10] I. Loshchilov and F. Hutter. Decoupled Weight Decay Regularization. *ICLR*, 2019.